Problema Comis-Voiajorului

Roșu Mihai Cosmin 343C1

Table of Contents

1. Introducere 2
1.1 Descrierea problemei 2
1.2 Restricții și precizări 2
2. Implementare serială 2
3. Implementări paralele 2
3.1 Pthreads 2
3.2 OpenMP 3
3.3 MPI 3
3.4 Implementări hibride 3
4. Compararea rezultatelor 4
4.1 Grafice de timp 4
4.2 Grafice de speed-up 7
5. Concluzii 10

1. Introducere

1.1 Descrierea problemei

Problema comis voiajorului are ca scop determinarea drumului de cost minim, care trece prin fiecare oraș o singură data, iar apoi se întoarce în orașul din care a pornit, pentru un set dat de orașe, fiind cunoscute costurile dintre orașe și punctul de plecare.

1.2 Restricții și precizări

Este considerat că între oricare două orașe există drum direct și că toate costurile drumurilor sunt strict pozitive.

2. Implementare serială

Pentru implementarea serială a fost ales un algoritmul naiv, care se bazează pe generarea tuturor drumurilor care pleacă din orașul precizat în input și trece o singură dată prin fiecare oraș, întorcându-se în orașul din care a plecat.

Deoarece fiecare oraș poate fi reprezentat unic printr-un număr, fiecare drum poate fi reprezentat printr-o permutare a orașelor. Așadar, algoritmul generează toate permutările orașelor și calculează pentru fiecare costul drumului reprezentat de permutare. În final, rezultatul este reprezentat de cel mai mic cost.

3. Implementări paralele

Implementările paralele se bazează pe implementarea serială. Diferența apare prin faptul că permutările sunt împărțite între thread-uri/procese după cel de-al doilea număr din permutare. Așadar, fiecare thread/proces este responsabil de permutările care au al doilea oraș în mulțimea orașelor atribuite lui.

Pentru determinarea drumului de cost minim, fiecare thread/proces calculează drumul de cost minim din permutările de care este responsabil, iar în final dintre rezultatele fiecărui thread/proces este ales cel mai bun drum.

3.1 Pthreads

Implementarea folosind Pthreads folosește următoarele formule pentru a determina mulțimea orașelor pentru fiecare thread:

$$start = \frac{ID * N}{P}$$

$$end = \min\left(\frac{(ID+1)*N}{P}, N\right)$$

unde ID = id-ul unui thread

N = numărul de orașe

P = numărul de thread-uri

Aceste formule sunt folosite de fiecare thread în parte pentru a-și calcula mulțimea orașelor folosite pentru a doua poziție din permutare.

Mulțimea orașelor este determinată de intervalul format de cele două numere **start** și **end**.

Pentru determinarea drumului de cost minim, fiecare thread reține intr-un array rezultatul lui, iar după terminarea procesării permutărilor, thread-ul principal parcurge array-ul de rezultate și determină cel mai bun drum.

3.2 OpenMP

Implementarea folosind OpenMP folosește macro-ul **parallel for** prin care thread-urile sunt pornite și compilatorul le asignează mulțimile de orașe.

Pentru determinarea drumului de cost minim, este folosită clauza **reduction** în cadrul macro-ului menționat anterior, cu operația **min**.

3.3 MPI

În cazul implementării folosind MPI, este folosită instrucțiunea **MPI_Scatterv** pentru a împărți mulțimile de orașe între procese. Instrucțiunea menționată necesită calcularea în prealabil a unor deplasamente și dimensiuni care indică porțiuni din mulțimea totală de orașe. Pentru acest calcul, procesul principal încearcă să împartă în mod egal orașele, iar dacă nu reușește, orașele rămase sunt împărțite din nou.

Pentru determinarea drumului de cost minim, fiecare proces cu excepția celui principal, trimite către cel principal cel mai bun drum determinat de el. În final, procesul principal alege costul minim dintre cele primite.

3.4 Implementări hibride

Implementările hibride, **MPI-Pthreads** și **MPI-OpenMP**, se bazează pe împărțirea orașelor între procese (MPI), iar apoi, fiecare proces împartea mulțimea primită între thread-uri (Pthreads/OpenMP).

Modul în care are loc împărțirea, atât între procese cât și între thread-uri, are loc în același mod ca la implementările simple.

4. Compararea rezultatelor

Pentru a putea compara implementările, au fost folosite teste cu [4-15] orașe, care au fost rezolvate folosind diferite numere de thread-uri/procese.

În cazul implementărilor hibride, a fost aleasă combinația de procese și thread-uri care a produs cel mai bun timp, pentru care produsul dintre numărul de procese și numărul de thread-uri este egal cu numărul de thread-uri/procese folosit de implementările simple.

Nume procesor	Intel Core i5-14600K
Nuclee fizice	14
Nuclee logice	20

Tabel 1. Specificații hardware

4.1 Grafice de timp

În primul grafic se pot observa și timpii de rulare pentru implementarea serială, în comparație cu timpii implementărilor paralele cu 4 thread-uri/procese. În următoarele grafice se află doar timpii implementărilor paralele întrucât timpii implementării seriale sunt aceeași, și datorită valorilor considerabil mai mari, nu se mai remarcă diferența între implementările paralele.

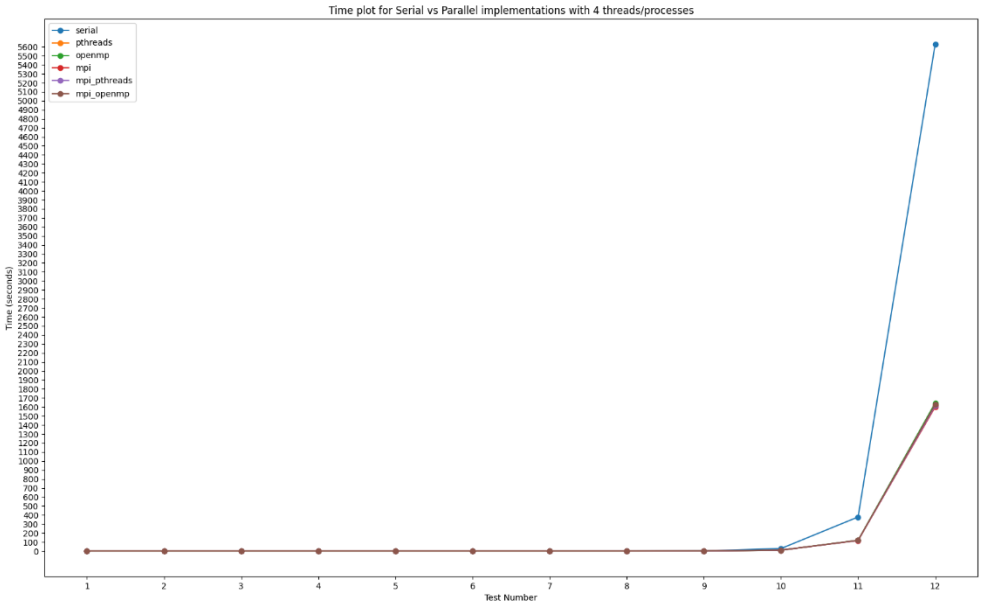

Figura 1. Timpii de execuție ai implementării seriale în comparație cu timpii implementărilor paralele folosind 4 thread-uri

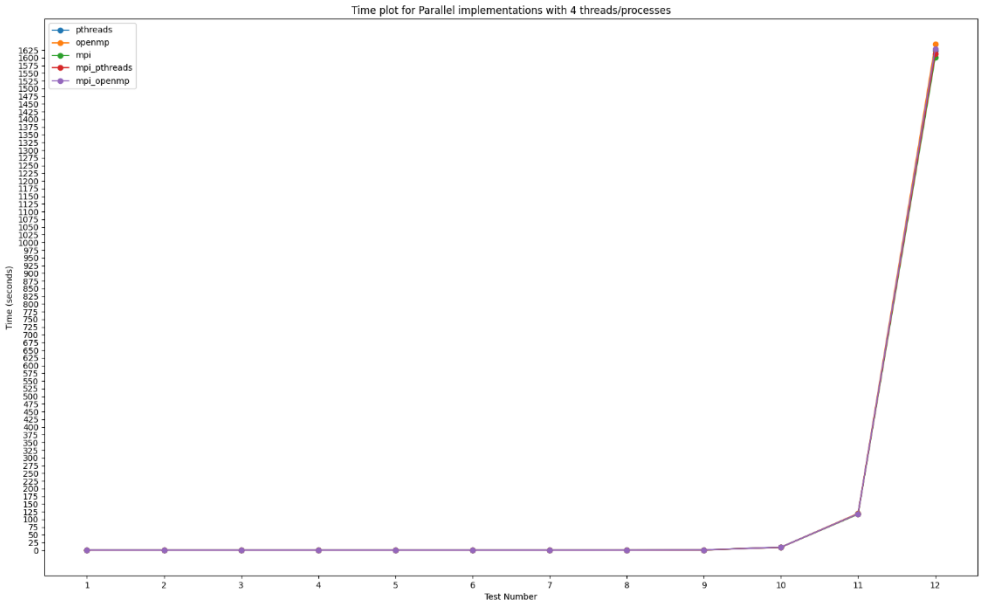

Figura 2. Timpii de execuție ai implementărilor paralele folosind 4 thread-uri sau procese

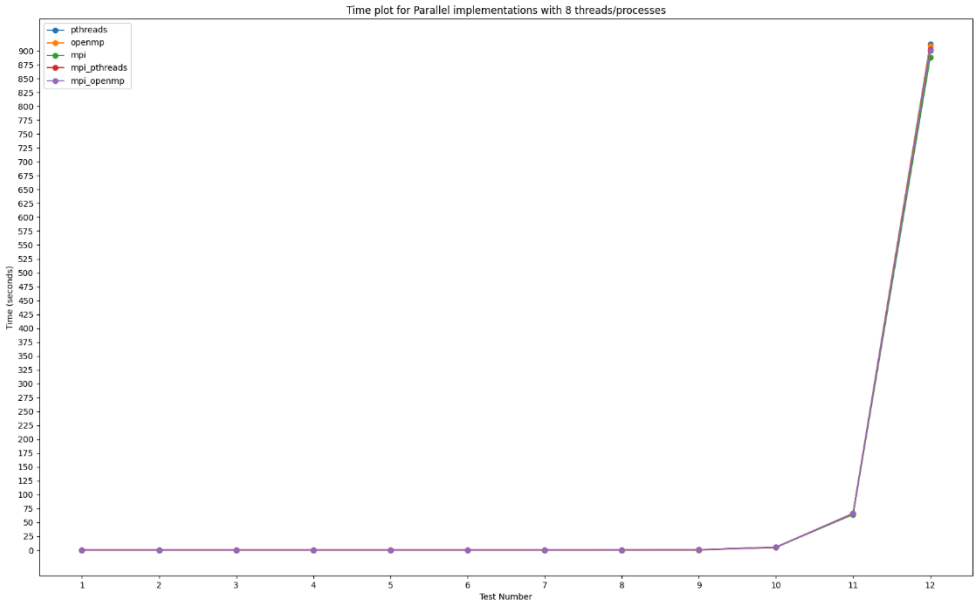

Figura 3. Timpii de execuție ai implementărilor paralele folosind 8 thread-uri sau procese

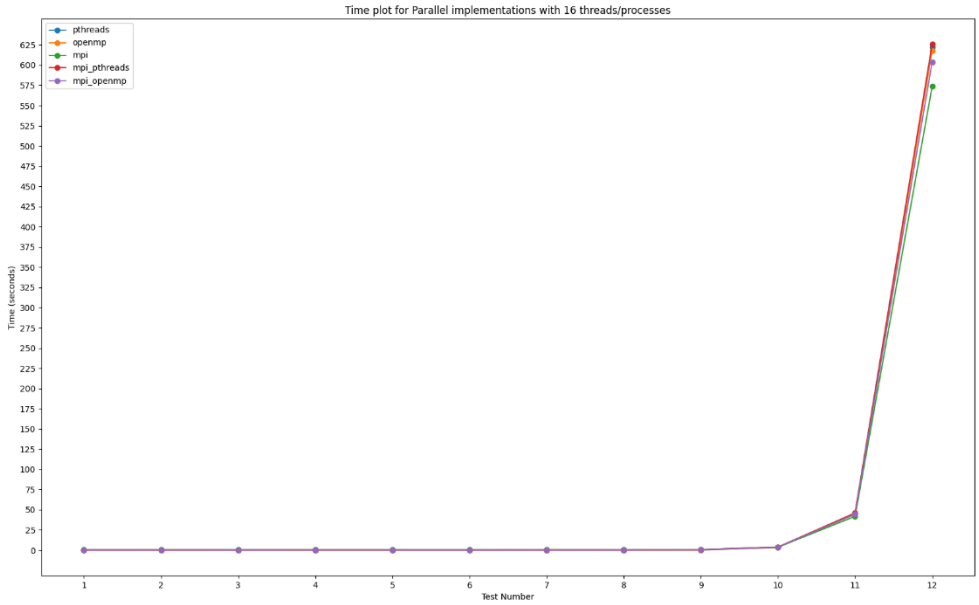

Figura 4. Timpii de execuție ai implementărilor paralele folosind 16 thread-uri sau procese

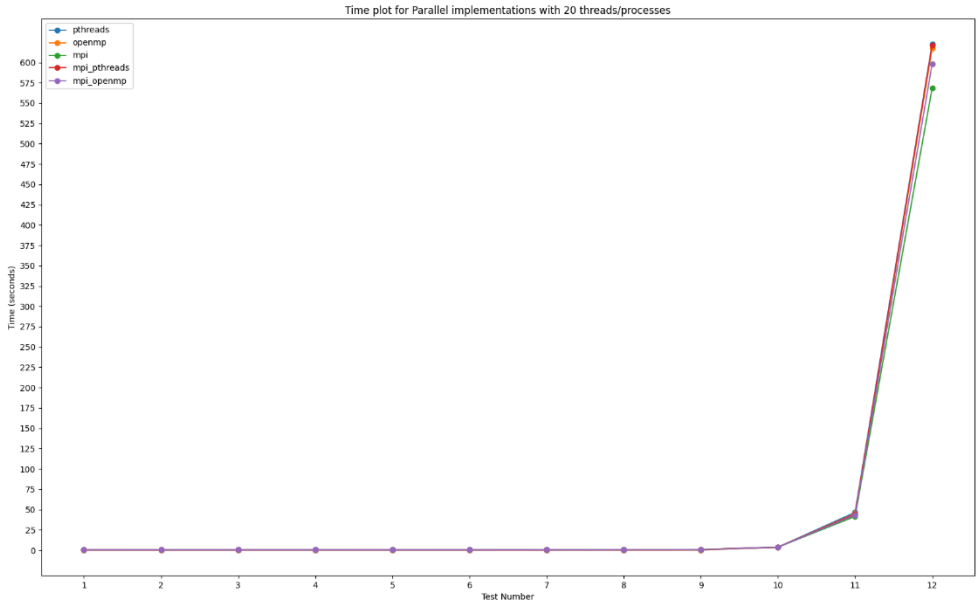

Figura 5. Timpii de execuție ai implementărilor paralele folosind 20 thread-uri sau procese

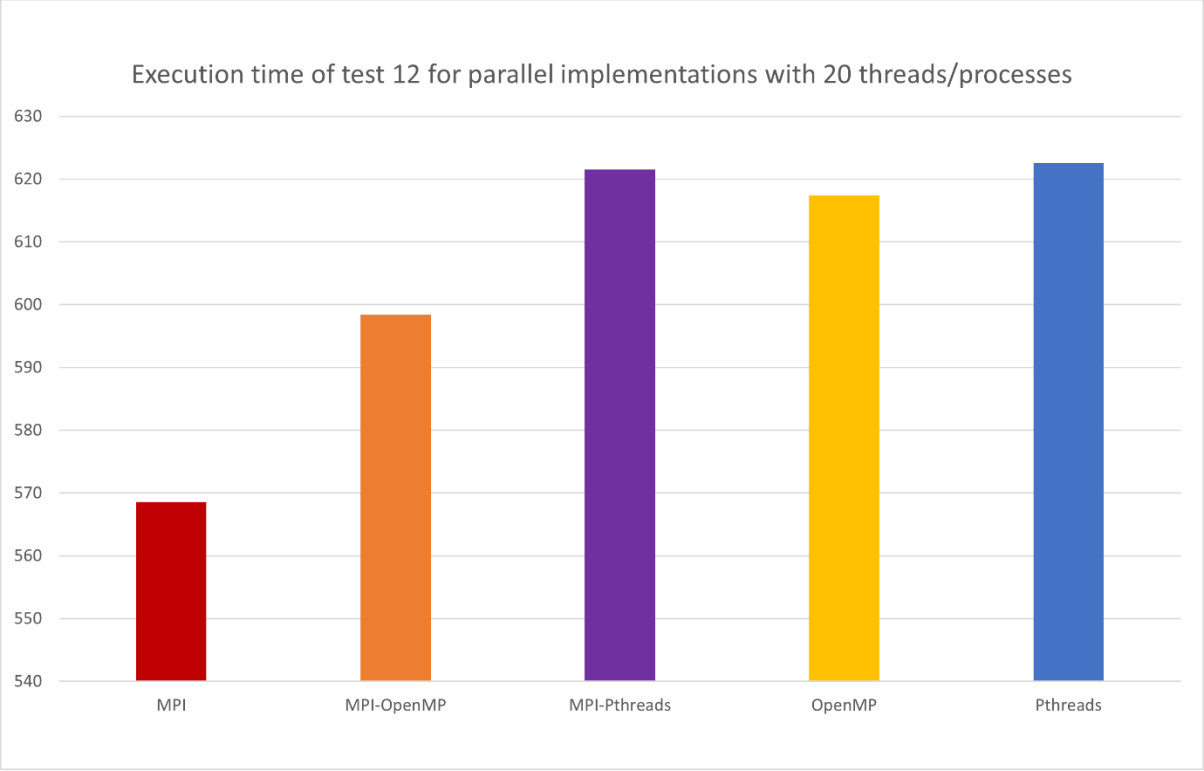

Figura 6. Comparație între timpii de execuție ai implementărilor paralele folosind 20 thread-uri/procese pentru testul 12 (cu 15 orașe)

4.2 Grafice de speed-up

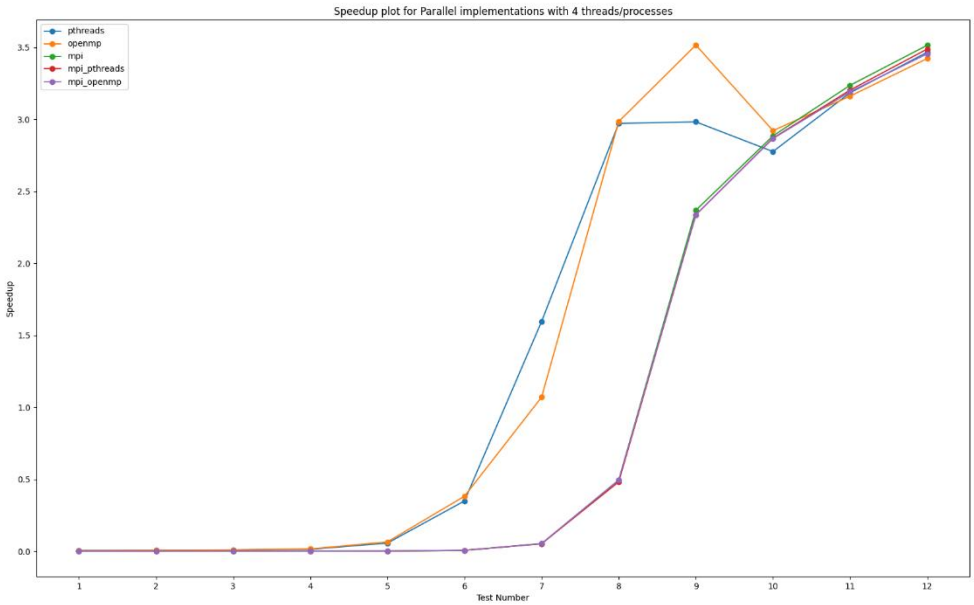

Figura 7. Comparație a speed-up-ului implementărilor paralele folosind 4 thread-uri/procese

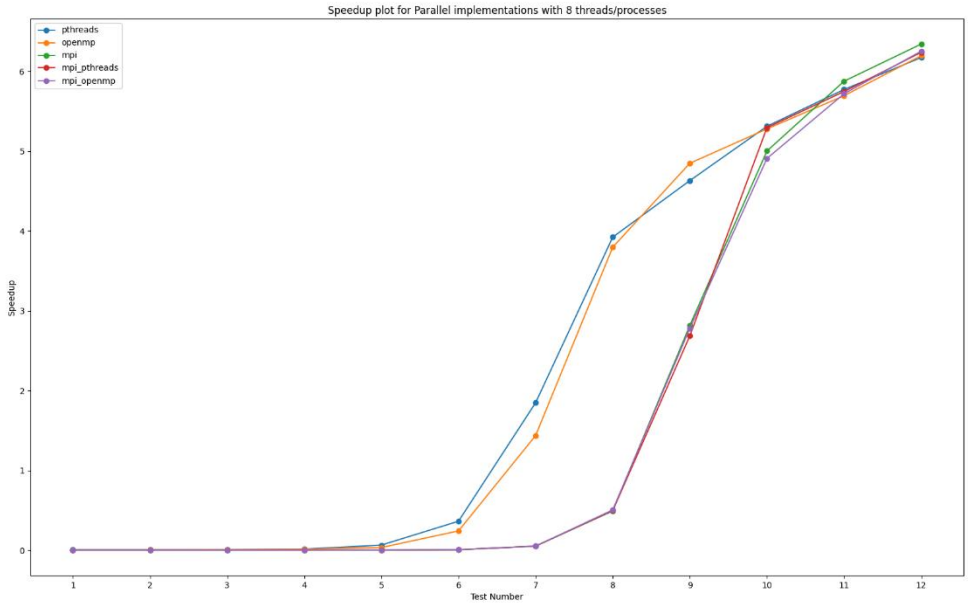

Figura 8. Comparație a speed-up-ului implementărilor paralele folosind 8 thread-uri/procese

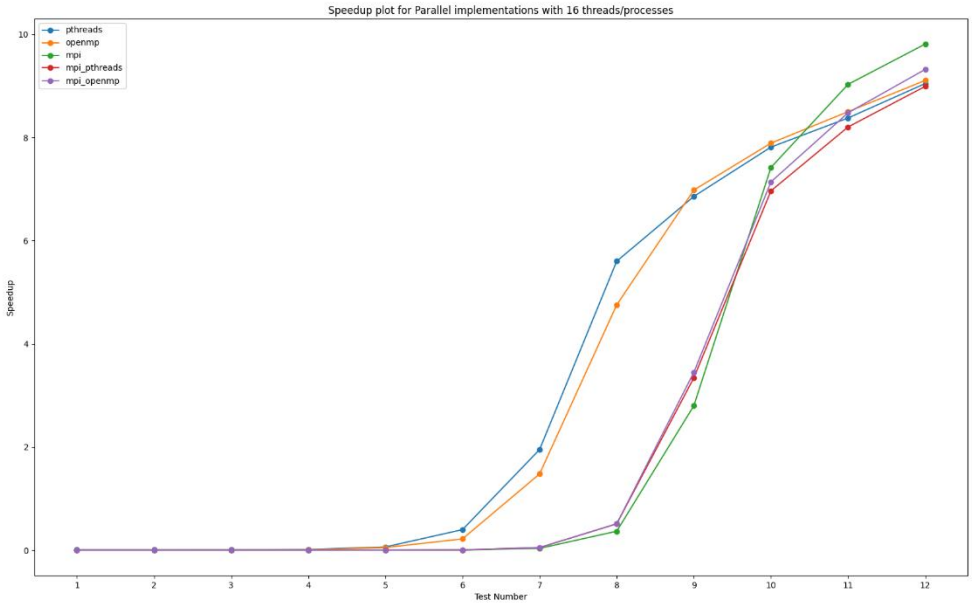

Figura 9. Comparație a speed-up-ului implementărilor paralele folosind 16 thread-uri/procese

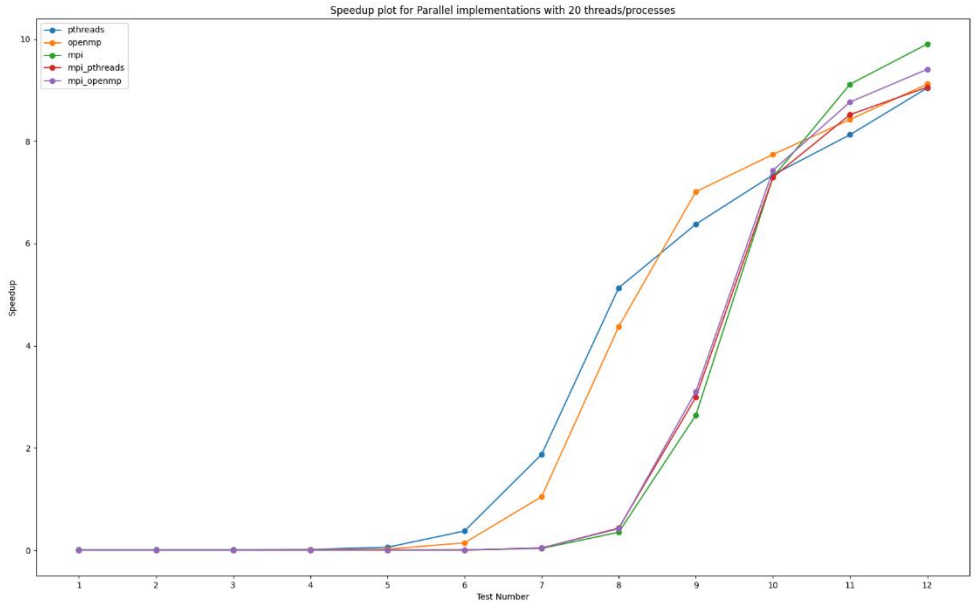

Figura 10. Comparație a speed-up-ului implementărilor paralele folosind 20 thread-uri/procese

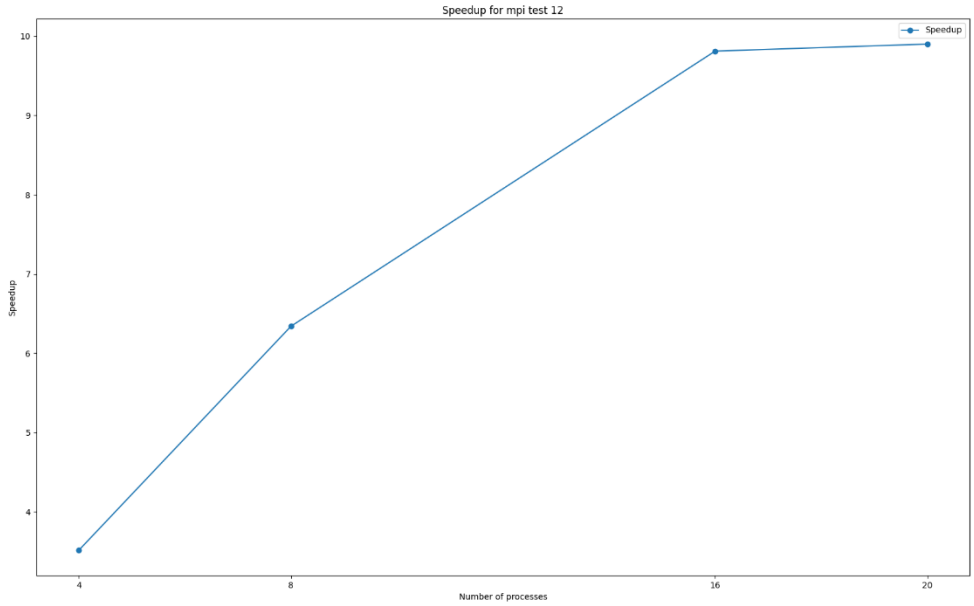

Figura 11. Speed-up-ul implementării MPI asupra testului 12 (cu 15 orașe) în funcție de numărul de procese

5. Concluzii

Din rezultatele prezentate mai sus se pot trage următoarele concluzii:

- Toate implementările paralele sunt mai eficiente decât implementarea serială;
- Pe testele mari (14/15 orașe) cea mai eficientă implementare este cea cu MPI, pentru orice număr de procese/thread-uri;
- Pe testele mici (sub 15 orașe) cea mai eficientă implementare este cea cu OpenMP, urmată de cea cu Pthreads care are rezultate foarte asemănătoare, pentru orice număr de thread-uri/procese;
- Pe testul 12 (cu 15 orașe) implementarea cu MPI este cu 5-10% mai eficientă decât restul;
- Implementările respectă Legea lui Amdahl, speed-up aplatizându-se atunci când numărul de thread-uri/procese depășește numărul de orașe.